

VulnHunter：融合程序分析与隔离式动态验证的仓库级漏洞审计框架

作者信息待补充

预印本草稿，2026 年 9 月 26 日

摘要

仓库级漏洞检测需要同时回答两类问题：目标函数是否存在可达的危险路径，以及该路径能否在具体项目环境中触发预期安全影响。仅依赖函数片段容易忽略跨文件约束，而把静态可疑路径直接视为真实漏洞又可能放大误报。本文介绍 VULNHUNTER，一种面向 Python 仓库的代理式审计框架。系统以目标函数及其对应仓库快照为起点，由主审计代理读取代码并按需调用 CodeQL、Semgrep 和基于代码属性图的 CodeBadger 工具；独立执行代理在每个任务专属的 Docker 容器中实施与假设相关的动态验证。执行结果通过带证据引用的追加式事实事件回传，最终由结构化报告明确区分实际观察到的影响与静态推断。我们给出任务隔离、证据汇聚、判定规则及成对评测协议，并对一份已完成的 RepoPairBench 历史运行日志进行复算：100 组漏洞-修复对的 200 个样例中，召回率为 66%，误报率为 30%，成对正确率为 39%。该日志产生于当前结构化结果实现之前，且未保存足够的模型与工具版本信息，因此这些数字是系统开发过程中的观察结果，不能用作当前版本的最终性能结论。本文进一步界定动态复现门槛、二值判定与环境失败之间的张力，为后续可复现实验提供明确的研究问题。

关键词：仓库级漏洞检测；大语言模型代理；程序分析；动态验证；证据可追溯；成对评测

1 引言

现实漏洞往往依赖目标函数之外的调用方约束、配置、权限检查与数据流。给定一个表面上包含安全函数的函数，仅依据局部文本无法判断检查是否绑定了正确的对象，也无法确定攻击者是否能够控制抵达危险操作的输入。DREA 将这一问题表述为由规划代理驱动的仓库探索，并在 RepoPairBench 上使用修复前后仓库快照评估检测与推理质量 [1]。其工作说明，自适应获取仓库上下文是比固定片段检索更有针对性的途径。

VULNHUNTER 面对一个相邻但不同的问题：当代理得到静态可疑路径后，如何把代码证据、工具结果和实际执行现象汇合为可核查的结论。静态分析工具可以揭示候选路径，却不自动证明运行时可达性；动态执行可以提供观察结果，却受到依赖安装、测试入口和容器环境的限制。本文研究一个把这两种证据接入同一审计流程的系统实现，并特别关注证据来源与结论强度是否匹配。

本文贡献是一个已实现的系统设计与一个有边界的初步测量。系统设计包含：面向函数目标的仓库探索和多工具调用；仅通过 Docker 工具操作隔离环境的执行代理；以事件而非可覆盖文本维护的事实黑板；以及要求阳性结论附带复现信息的结构化结果契约。测量部分复算本地完整历

史运行的样例指标与成对指标，同时公开指出其与当前代码版本之间的差异。我们不把该历史运行与 DREA 论文中的不同模型配置做因果比较。

2 问题定义与研究问题

对第 i 个修复提交，设 R_i^- 为修复提交的父版本仓库， R_i^+ 为修复后的仓库； f_i^- 和 f_i^+ 为数据集给出的对应目标函数。评测输入为 (R_i^s, f_i^s) ，其中 $s \in \{-, +\}$ 。期望标签分别为 $y_i^- = 1$ 与 $y_i^+ = 0$ 。审计系统输出二值标签 $\hat{y}_i^s$ ；若输出漏洞结论，还应给出受影响位置、触发条件、复现步骤、观察到的影响和证据引用。

本文区分三个层次：线索是静态工具或代码阅读指出的可疑操作；推断是根据调用链、输入来源和约束形成的漏洞假设；观察是在隔离环境中执行并记录的行为。三个层次之间不能直接划等号。围绕该区分，我们提出下列研究问题：

1. **RQ1**：多种静态分析工具与目标导向的仓库阅读能否帮助定位待验证的触发路径？
2. **RQ2**：隔离式动态验证和追加式证据记录能否改善阳性报告的可复核性？
3. **RQ3**：严格的动态确认门槛在召回率、误报率与无法验证样例之间产生何种权衡？

现有完整日志只支持对二值标签及成对结果作描述性回答；RQ1–RQ3 的因果判断仍需对照和消融实验。

3 系统设计与实现

3.1 任务输入与仓库快照

VULNHUNTER 支持整仓审计和函数目标审计。对函数目标，输入含仓库地址、提交引用、文件相对路径及目标函数代码。评测服务对漏洞版检出修复提交的父提交，对修复版检出修复提交本身；每个样例在独立目录中执行，并在进入代理前检查目标文件是否存在。主代理读取目标文件及必要的调用上下文，但最终判定范围限定在所给函数。该约束避免把同一仓库中的其他漏洞误记为目标函数命中。

3.2 静态线索获取与假设更新

主代理可使用只读文件浏览能力，以及 CodeQL、Semgrep 和 CodeBadger 提供的分析工具。CodeQL 的路径查询可呈现源与汇点之间的数据流 [5]；Semgrep 通过规则匹配与污点分析提供候选发现 [6]；CodeBadger 在本项目中提供基于 Joern 代码属性图的查询接口，代码属性图把多种程序结构纳入统一图表示 [4]。这些工具的返回值被当作待审查证据，而非自动投票器。主代理据此检查输入控制、跨函数传播、保护条件和危险操作，再决定是否交由执行代理验证。系统没有固定要求每个任务调用全部工具，因此工具使用频率需要在后续实验中由轨迹统计。

3.3 隔离式动态验证与证据黑板

执行代理拥有 Docker MCP 工具和 `append_blackboard`，不继承主代理的宿主文件工具。主代理发送与当前漏洞假设相关的验证计划；执行代理在样例容器中运行最小必要步骤，并以

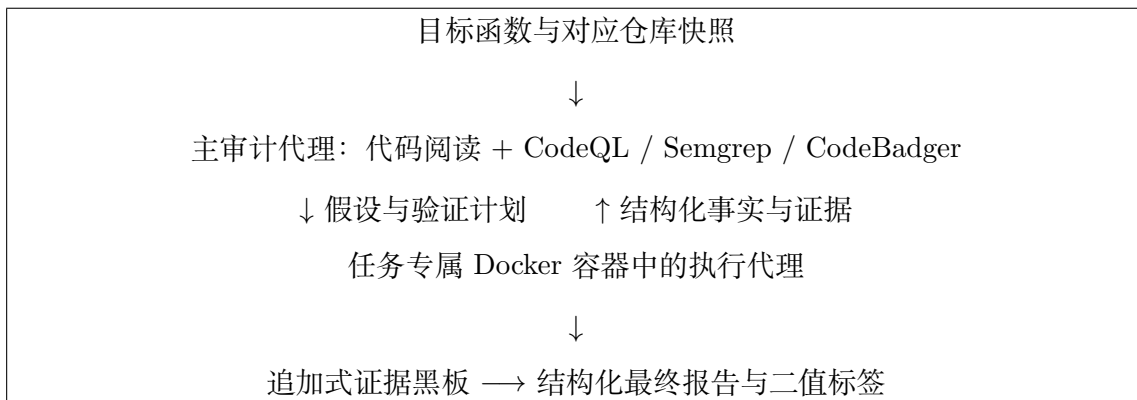


图 1: VULNHUNTER 的审计数据流。执行代理只通过容器工具交互；主代理读取事实黑板中的已记录证据。

`confirmed`、`unconfirmed` 或 `inconclusive` 描述执行状态。只有实际触发攻击并观察到预期安全影响，才允许把动态状态写为 `confirmed`。工具异常被包装成带错误状态的工具消息，使代理可以看到失败原因，而不是把异常返回误认为有效证据。

事实黑板的基本单元为 $(id, facts, evidence)$ 。其中事实为追加列表，证据记录含类型、引用和可选摘录。状态合并按事件标识去重，并在发现同一标识对应不同内容时拒绝静默覆盖。主代理每轮从事件集合重建黑板文本；这使执行代理的独立快照或重试不会通过覆盖一段旧文本抹除已记录事实。该机制保证的是记录一致性，并不保证模型书写的每条事实本身真实；证据质量仍需单独审查。

3.4 结论契约与资源生命周期

当前输出模式把 `verdict` 限定为 0 或 1。阳性必须附带漏洞报告，含摘要、位置、前置条件、复现步骤、预期与观察到的影响、验证状态以及至少一条证据；阴性不得附带阳性报告。最终整理提示进一步要求：仅有静态推断、理论 PoC 或未复现的尝试均不能产生阳性标签。这一门槛使报告的阳性语义较强，却把复现失败和确实无漏洞都压进二值输出，构成当前实现的重要限制。

任务队列允许并发审计和评测，每个样例拥有独立检出目录、容器名称及结果路径。样例结束、失败或取消时，系统在 `finally` 路径清理容器与检出目录；评测日志按状态保留，便于恢复中断任务。资源隔离是运行正确性的条件，不能视为漏洞检测准确性的证据。

4 评测协议

4.1 数据集与指标

本项目导入 DREA 发布的 RepoPairBench 100，包含 100 组 Python 漏洞-修复对，共 200 个实例，覆盖 48 类 CWE[1]。本地评测记录只保存数据集元数据；运行时根据仓库 URL 和提交引用检出两侧代码。样例输入使用目标路径和函数代码，标签计算使用元数据中的漏洞版/修复版信息。修复提交及其父提交的配对方式使同一项目的前后版本可以一起评估。

设混淆矩阵为 TP、FN、FP、TN，定义

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}, \quad (1)$$

$$F_1 = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}, \quad J = \text{Recall} - \text{FPR}. \quad (2)$$

成对正确率要求同一对的漏洞版判阳性、修复版判阴性：

$$\text{PC} = \frac{\sum_i \mathbf{1}[\hat{y}_i^- = 1 \wedge \hat{y}_i^+ = 0]}{N_{\text{complete pairs}}}. \quad (3)$$

未运行或取消的样例不进入混淆矩阵；已运行但无法解析标签的样例应单独统计，并在二值指标中计为相应侧的漏判或误报。若成对的任一侧无有效标签，则该对不进入成对正确率分母，并须报告覆盖量。这样的分母定义与系统当前评测模块一致，也防止因过滤失败样例而虚增成绩。

4.2 历史完整运行的复算

评测日志中存在一项从 2026 年 9 月 25 日至 26 日执行的完整运行¹。我们按每个样例标识取日志中的最终状态，核对得到 200 个 `done` 样例和 100 个双侧完整的修复对。该运行输出使用旧版自由文本结论及 `vulnerable/benign` 标签；当前代码已改为结构化 `AuditAssessment`，因此表 1 只描述这次历史运行。

表 1: RepoPairBench 历史完整运行的日志复算 (100 对、200 个样例)。

项目	数值	项目	数值
TP / FN	66 / 34	FP / TN	30 / 70
召回率	66.0%	误报率	30.0%
精确率	68.8%	F_1	67.3%
准确率	68.0%	Youden J	36.0 个百分点
成对正确	39 / 100	双侧均阳性	27 / 100
双侧均阴性	31 / 100	标签相反	3 / 100

从表 1 可以观察到，66 个漏洞版本被判阳性，同时 30 个修复版本也被判阳性。成对正确率 39% 低于单样例准确率 68%，说明在修复前后都给出正确结论比在随机单侧答对更严格。双侧均阳性的 27 对和双侧均阴性的 31 对分别提示过度报警与保守漏报两种错误形态。这些只是标签分布的解释，尚不能归因于某个工具或验证门槛。

4.3 需要补齐的对照实验

发表前应固定代码提交、模型名称及版本、推理参数、工具版本、容器镜像、并发数和运行时间，并在相同样例与模型条件下至少比较：仅函数输入、仅仓库阅读、仓库阅读加静态工具、完整 VULNHUNTER。对于动态验证，应另外报告已复现、未复现、环境失败和未执行的数量，以及阳性报告中证据引用的可核查比例。每组至少重复运行并给出波动范围，避免把一次代理轨迹当成确定性结果。若与 DREA 对比，必须统一骨干模型、预算、目标函数和数据集处理；其论文公布的指标只能作为外部背景，不应与表 1 直接计算提升幅度。

¹运行标识：eval-1790344118836-1。

5 讨论与效度威胁

严格阳性与不确定性。 运行环境缺依赖、入口无法建立或攻击未触发时，“尚未确认”不等于“漏洞不存在”。当前结果契约要求二值答案，且只允许动态成功产生阳性，可能把环境失败计成假阴性。后续版本应把 `inconclusive` 作为独立结果，并同时报告覆盖率与只在可判定样例上的检测指标。改变标签语义后必须重新运行基准，不能沿用表 1 的数字。

版本与可复现性。 历史运行日志没有完整保存当时的模型标识、工具版本、提示词版本或代码提交；其自由文本输出也与目前的结构化结果不同。因此本稿不把该结果归属给当前实现，不宣称其可由现有源码原样复现。当前代码可重建的只是评测协议和评分规则，而非当时的推理环境。

基准与证据泄露。 修复对标签来自公开提交。系统没有把 CVE/CWE 元数据直接写入目标函数提示，但仓库中的提交历史和修复线索可能被工具间接读取；评测应保存工具调用轨迹，并审查是否通过补丁或提交信息获得答案。RepoPairBench 聚焦 Python 修复对，结果不能直接推广至其他语言、未披露漏洞或真实持续审计场景。

工具与观察误差。 CodeQL、Semgrep 与代码属性图查询可能因规则覆盖、解析失败或环境故障遗漏路径。动态 PoC 也可能只验证一个变体，或在与生产环境不同的依赖版本中表现不同。黑板记录的是代理报告的证据引用，不能替代人工或自动化的证据审计。报告中应分别标示观察到的事实与推断的安全影响。

6 相关工作

ReAct 将推理与环境动作交替组织，为可调用工具的语言模型代理提供了通用范式 [2]。SWE-agent 展示了仓库导航与命令执行接口对软件工程代理的重要性 [3]。DREA 采用规划代理与轻量探索代理，并提出 RepoPairBench 和推理正确性评估 [1]。VULNHUNTER 与其共享仓库上下文和成对评测的任务设置，但当前实现采用同一模型驱动主审计与容器执行代理，目标是连接静态线索、动态观察和可追溯报告；我们没有证据表明它继承了 DREA 的成本优势。CodeQL、Semgrep 和代码属性图分别提供不同形式的程序分析能力 [5, 6, 4]，在本文框架中它们是生成与核查候选证据的工具，而不是最终标签的独立来源。

7 结论

本文给出 VULNHUNTER 的仓库级审计流程、证据黑板及隔离式动态验证设计，并对一次完整但版本信息不足的历史评测进行透明复算。下一阶段的关键工作是固定实验配置、引入明确的不确定结果、验证报告证据，并在同条件对照与消融实验中检验各组件的实际贡献。在这些实验完成前，历史成对正确率 39% 仅表明该次运行的观察结果。

参考文献

- [1] Mingyang Sun and Guozhu Meng. DREA: Decoupled Reasoning and Exploration Agents for Repository-Level Vulnerability Detection. arXiv:2607.13439, 2026.
<https://arxiv.org/abs/2607.13439>.
- [2] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023. <https://arxiv.org/abs/2210.03629>.
- [3] John Yang, Carlos E. Jimenez, Alexander Wettig, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793, 2024.
<https://arxiv.org/abs/2405.15793>.
- [4] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. Modeling and Discovering Vulnerabilities with Code Property Graphs. IEEE Symposium on Security and Privacy, 2014.
<https://www.ieee-security.org/TC/SP2014/papers/ModelingandDiscoveringVulnerabilitieswithCodePropertyGraphs.pdf>.
- [5] GitHub. Exploring data flow with path queries.
<https://docs.github.com/en/code-security/how-tos/find-and-fix-code-vulnerabilities/scan-from-vs-code/explore-data-flow>.
- [6] Semgrep. Writing Semgrep rules. <https://semgrep.dev/docs/writing-rules/rule-ideas>.